Department of Computer Science & Engineering • IIT Bombay

CS6008: Network Security

Assignment 2: DNS Security

Weight: 10% of Course Grade
Submission: Git repo (`.zip`/`.tar.gz`) with code, configs & PDF report

Team Size: Individual or Pair
Deadline: 23 October 2026

Overview

DNS answers are, by default, unauthenticated cleartext: anyone on the path between a client and its nameservers can read a query and forge the reply. In this assignment you build a small, self-contained DNS hierarchy in containers, attack it from an on-path position, and then add defenses one layer at a time to see which attacks each one actually stops.

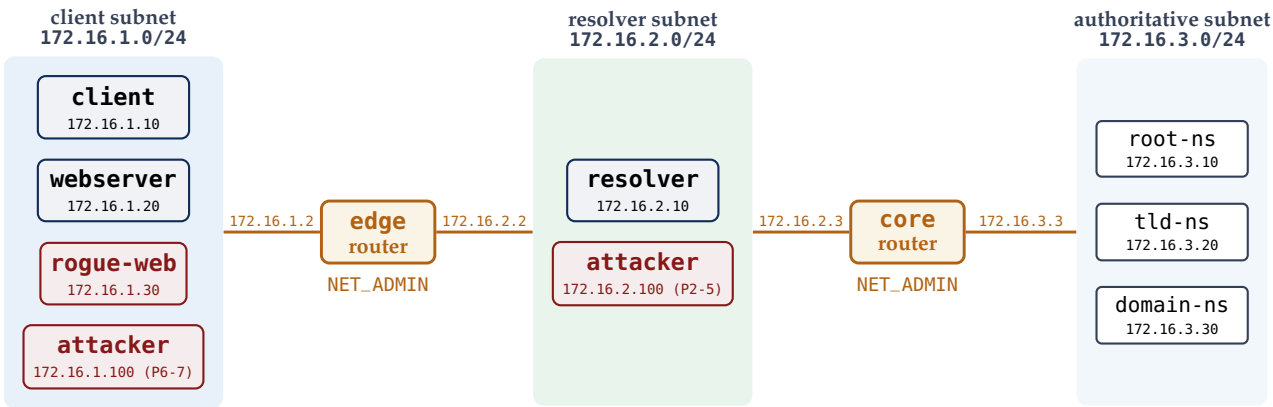
This assignment is split into seven phases: (1) deploy plain hierarchical DNS, (2) forge responses to poison the resolver, (3) add DNSSEC and repeat the attack, (4) attempt a signature-stripping downgrade, (5) walk a signed zone, (6) move the attacker to the client's last mile, and (7) encrypt that last mile with DoT/DoH. Every phase has something to deploy, something to verify, and questions to answer in your report; the outcome of each attack is for you to observe and explain.

Environment & Tooling

- **Orchestration:** Docker & Docker Compose. All nodes run on isolated Docker bridge networks; nothing touches the real Internet or public DNS roots.
- **DNS daemon:** BIND 9.20.x on every nameserver and on the resolver.
- **Inspection:** `dig`, `delv`, `kdig`, `curl`, and Wireshark / `tshark` for packet capture.
- **PKI:** OpenSSL, for the private CA and the resolver's TLS certificate (Phase 7).
- **Routing:** `iproute2` for IP forwarding on the routers and static routes on every node (**no NAT**).
- **Attack tooling:** every attack program — the on-path interceptors that forge or strip DNS (Phases 2–4 and 6–7) and the Phase 5 zone-walker — must be written in **C or C++** using **`libpcap`**, which both captures packets and injects them (`pcap_inject` / `pcap_sendpacket`). High-level spoofing or DNS frameworks (Python/Scapy, dnspython, and similar) are **not permitted**. The inspection tools above may still be used to verify your results.

Components

Every node is a container with a fixed address in exactly one of three subnets (Docker bridges) — except the two routers, which each have an interface in two subnets and are the *only* path between them. Use the addresses below in every phase. (Docker reserves `.1` on each bridge for the host, so nothing is placed there.)



Container	Subnet	Address	Role
`client`	client	`172.16.1.10`	End host issuing lookups and web requests
`webserver`	client	`172.16.1.20`	Serves the legitimate `example.com` site over HTTP
`rogue-web`	client	`172.16.1.30`	Adversary's webserver with the defacement page (Phase 2+)
`resolver`	resolver	`172.16.2.10`	Recursive caching resolver; validates DNSSEC from Phase 3
`root-ns`	authoritative	`172.16.3.10`	Authoritative nameserver for the root zone (`.`)
`tld-ns`	authoritative	`172.16.3.20`	Authoritative nameserver for the `.com` zone
`domain-ns`	authoritative	`172.16.3.30`	Authoritative nameserver for `example.com`
`edge-router`	client	`172.16.1.2`	Routes client ↔ resolver traffic (no NAT)
	resolver	`172.16.2.2`	
`core-router`	resolver	`172.16.2.3`	Routes resolver ↔ authoritative traffic (no NAT)
	authoritative	`172.16.3.3`	
`attacker`	resolver	`172.16.2.100`	On-path adversary on the resolver subnet (Phases 2–5)
	client	`172.16.1.100`	Same adversary, moved to the client's last mile (Phases 6–7)

Privilege policy — read before you start

The **routers** are the only containers that *forward* traffic: they enable IP forwarding (**no NAT**) and hold the routing and attack-diversion routes. Every other node is configured with **static routes** that reach other subnets through the routers, so it needs `NET_ADMIN` only to install those routes — it must **not** forward, bridge, or NAT. The `attacker` additionally holds `NET_RAW` for its `libpcap` capture and injection. Isolation comes from the topology: no subnet has a direct link to another, so every cross-subnet packet passes through a router you control.

Submission Requirements

Submit **one Git repository**, archived as a single `.zip` or `.tar.gz` (keep the `.git` history in it), by the deadline. Inside, a `phase1/` through `phase7/` directory, any shared material under `common/`, and `report.pdf` at the top.

- **Each phase directory** holds everything that phase needs: its own `compose.yml`, zone and daemon configs, router and diversion scripts, attack source with a build command, and a `README` listing the exact commands to deploy, attack, and verify. A grader on a clean checkout, with only the host tools listed above, must be able to reproduce your results from that `README` alone.
- **Report** (`report.pdf`): one section per phase. For each, give your design choices, the verification and attack evidence (terminal output and annotated packet captures), and your answers to that phase's *Report* questions — not only what happened, but why.
- **Git hygiene:** commit incrementally with meaningful messages. Commit sources, configs, and scripts only — **not** artifacts your scripts regenerate (compiled binaries, signed zones, built Docker images, and the like); the grader builds them from your sources.
- **Generative AI:** if you used it, add an appendix listing your prompts and how you verified the output.

Marking — 100 marks

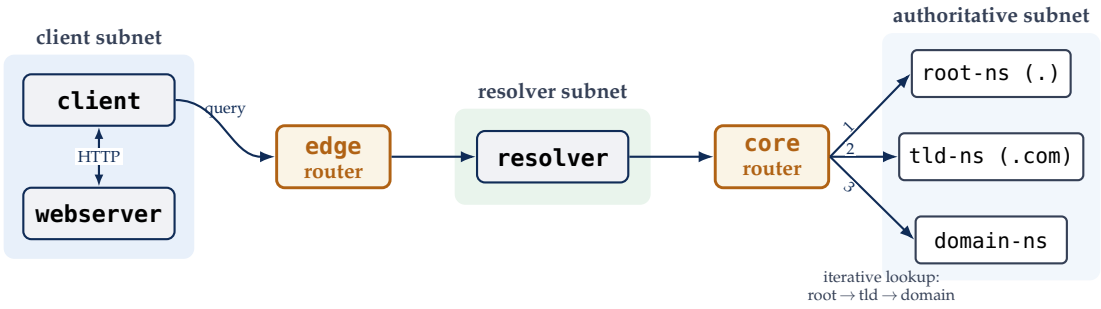
The seven phases carry the marks shown in their titles (**95** in all); the remaining **5** are for **overall report quality** — clarity, well-chosen evidence, and depth of reasoning.

Before you begin

You need a standard Docker install (Engine or Desktop) with Compose and Buildx, able to run containers with added capabilities (`NET_ADMIN` for the routers, `NET_RAW` for the attacker's `libpcap`) — rootless or locked-down Docker may refuse them. Developed and tested on **Linux** (Docker Desktop on macOS/Windows should work too); ∼4 cores, ∼8 GB RAM, and ∼15 GB free disk is comfortable. If anything is unclear or misbehaves, reach out to the TAs. All the best!

Phase 1 — Building the DNS Hierarchy [10 marks]

Stand up the three-tier hierarchy of the Components table and prove it is both correct and isolated. There is no attacker; this is the baseline every later phase builds on.



1.1 Deploy

- Place every container at its address from the Components table. `root-ns` delegates `com.` to `tld-ns`; `tld-ns` delegates `example.com.` to `domain-ns`; `domain-ns` serves `A` records for `example.com` and `www.example.com` pointing at `webserver` (`172.16.1.20`).
- The `resolver` is recursive with DNSSEC validation **off**.
- `webserver` serves a recognizable page for `example.com` over HTTP.
- **Cross-subnet reachability.** The two routers forward between their subnets with IP forwarding and **no NAT**; every other node reaches an off-subnet address through the router on its segment via a **static route**. Install the routes each node needs:
  - `client`: the resolver subnet (`172.16.2.0/24`) via `edge-router` at `172.16.1.2`.
  - `resolver`: the authoritative subnet (`172.16.3.0/24`) via `core-router` at `172.16.2.3`, and the client subnet (`172.16.1.0/24`) via `edge-router` at `172.16.2.2`.
  - each nameserver: the resolver subnet (`172.16.2.0/24`) via `core-router` at `172.16.3.3`.

  Addresses stay real end to end — root hints and glue name the nameservers at their `172.16.3.x` addresses. No node except the two routers forwards traffic.

1.2 Verify

- From `client`: `dig example.com` returns `172.16.1.20`, and `curl http://example.com` returns the real page.
- From `client`, a query sent directly to `172.16.3.10` (`root-ns`) gets no answer — the authoritative subnet is not directly reachable.
- On `core-router`, capture one full resolution and identify the root, `com`, and `example.com` exchanges.

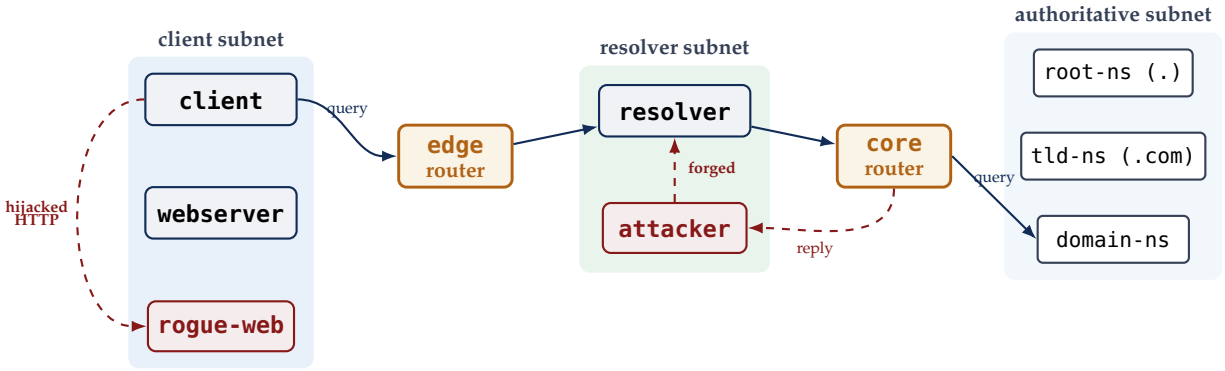
1.3 Report

- Show the resolved address and fetched page, with the commands that produced them, and the isolation check above.
- From your capture, trace the Root → TLD → Domain referral. For each message, what is visible on the wire (names, identifiers, answers)?
- From your capture (and `ip route`), show how the `resolver`'s query reaches `root-ns`: the static route it uses and the router it traverses to the authoritative subnet.
- Given what the capture exposes, what could a node on the resolver's path do to this resolution? You act on it in Phase 2.

> **Deliverables**
>
> `phase1/` (compose, zone and daemon configs, router and static-route scripts) · terminal logs of resolution, fetch, and the failed direct query · annotated capture of the referral chain · report answers.

Phase 2 — On-Path Response Forgery [15 marks]

Place an adversary on the resolver's subnet and poison the resolver's cache for one name, redirecting the client's web traffic to a rogue server — while all other resolution continues undisturbed.



2.1 Deploy

- Carry over the full Phase 1 deployment. Add the `attacker` (`172.16.2.100`) on the resolver subnet and keep `rogue-web` (`172.16.1.30`) reachable by the client.
- Divert the resolver's inbound responses through the attacker by **routing**: add a route on the `core-router` that sends that reply traffic via the `attacker`, which relays it on. The diversion is a route on the router only — the resolver and nameservers keep their Phase 1 configuration untouched.
- Write the interceptor in C/C++. It relays all DNS traffic unchanged except for `example.com`, for which it injects an `A` record pointing at `rogue-web` and ensures the resolver accepts the forged answer over the genuine one.

2.2 Verify

With forging disabled, the interceptor relays transparently: `client` still resolves `example.com` to `172.16.1.20` and fetches the real page.

2.3 Attack

Enable forging, flush the resolver cache, then from `client` resolve `example.com` and fetch the site. The goal: `example.com` resolves to `172.16.1.30` and `curl` returns the rogue page, while `www.example.com` still resolves to `172.16.1.20`.

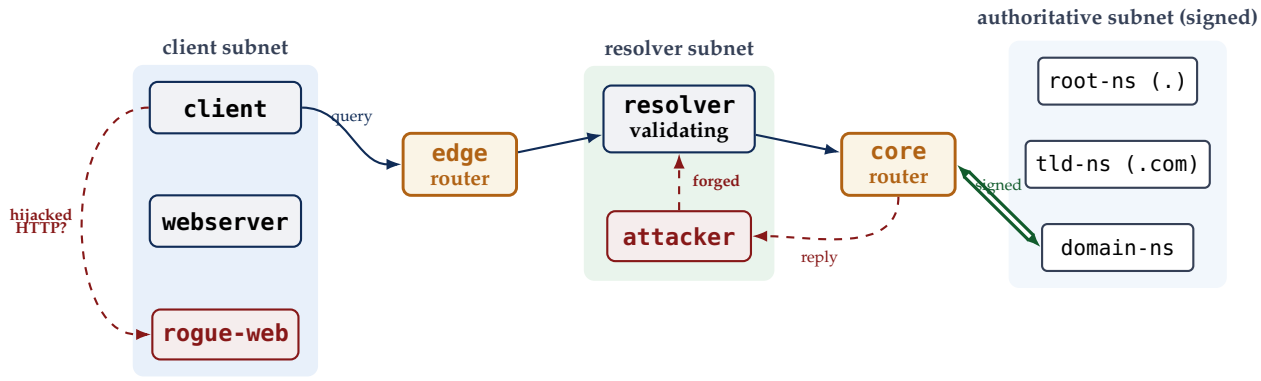
2.4 Report

- Show what the resolver cached and what the fetch returned, with logs and an annotated capture of the interception and injection.
- Explain *why* the attack succeeds: what property of plain DNS, and what advantage does your on-path position give over an attacker who cannot see the query?
- Explain how your relay leaves non-target names untouched.

> **Deliverables**
>
> `phase2/` (extended compose, attacker C/C++ source + build, diversion scripts) · logs and captures of the attack and of an unaffected non-target lookup · report answers.

Phase 3 — DNSSEC Chain of Trust [15 marks]

Add a cryptographic chain of trust and re-run the Phase 2 attack *unchanged*. The adversary keeps every advantage it had; only the zones and the resolver change.



3.1 Deploy

- On each authoritative server, sign its own zone: `root-ns` signs the root (`.`), `tld-ns` signs `com`, and `domain-ns` signs `example.com`.
- Chain them with `DS` records in the *parent*: publish `com`'s `DS` in the root zone (on `root-ns`) and `example.com`'s `DS` in the `com` zone (on `tld-ns`), giving an unbroken chain of trust from the root key down to `example.com`.
- On the `resolver`, install your root `KSK` as the single trust anchor and enable DNSSEC validation. Leave its root hints (to `root-ns`) as in Phase 1.
- Leave the `attacker` and its Phase 2 binary unchanged.

3.2 Verify

Before attacking, with no interceptor in the path, inspect a `dig +dnssec example.com` answer for the `ad` (authenticated-data) flag and a valid `RRSIG`, and validate the full chain with `delv` pointed at your root key as its trust anchor.

3.3 Attack

Flush the cache, trigger the client lookup and fetch exactly as in Phase 2.

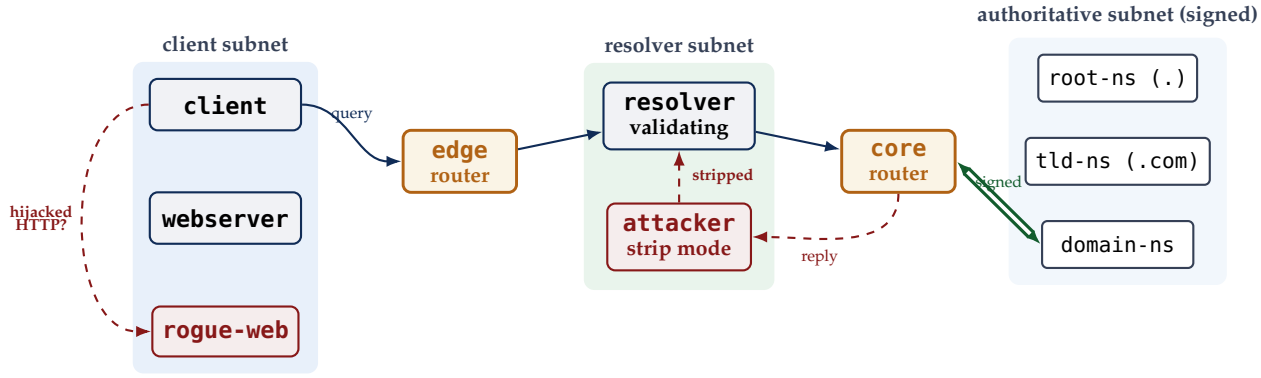
3.4 Report

- Report what the resolver does with the forged response and what the client's fetch returns, with logs and a capture. How does this differ from Phase 2, and what specifically does the resolver check to produce that outcome?
- Compare the attacked exchange on the wire with your Phase 1 capture. What can the on-path attacker still read? What does DNSSEC therefore protect, and what does it not?
- State your signing-algorithm choice and its trade-offs, and whether any private key belongs in the submitted repository.

> **Deliverables**
>
> `phase3/` (signing scripts, signed zones, delegation chain, resolver trust anchor) · evidence of a healthy signed hierarchy · logs/captures of the re-run attack · report answers.

Phase 4 — Signature Stripping & Downgrade [15 marks]

Forging a signed answer failed in Phase 3. Now make the *signed* zone look *unsigned*, so the plain forgery of Phase 2 is accepted again. The attacker may rewrite anything in transit but holds no private keys.



4.1 Deploy

- Reuse the Phase 3 signed hierarchy and validating resolver, unchanged.
- Extend the interceptor with a *strip* mode that, for `example.com`, removes DNSSEC records from the response and substitutes a forged `A` pointing at `rogue-web`.

4.2 Verify

Before stripping, `client` still resolves `example.com` as an authenticated answer.

4.3 Attack

- **Run A.** Strip the `RRSIG` and `DNSKEY` records for `example.com` — and, for a fuller downgrade, the `DS` record (and its `RRSIG`) from the `com` referral too — and inject the forged `A`. Record what the resolver returns to the client.
- **Run B.** Change the deployment so `example.com` stays *fully signed* but its parent `com` publishes **no** `DS` for it — the operator signed the child yet never gave the `DS` to the parent — and mount the *same* strip-and-forge attack. With no secure link into `example.com`, the still-root-anchored resolver treats it as unsigned, ignores its signatures, and accepts your forged record. The goal: `client` receives `172.16.1.30` for `example.com` from the validating resolver.

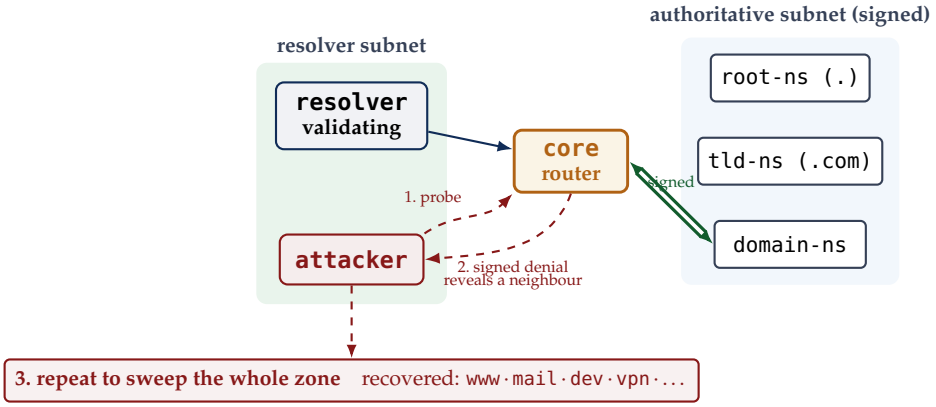
4.4 Report

- Report both runs with logs and captures. In Run A, what happens when the signatures — and even the `DS` — are stripped on the wire, and why can't the attacker make the zone look unsigned? In Run B, what single change did you make, and why does it reopen the attack?
- From the contrast, state where downgrade resistance actually comes from, and the exact operator mistake that forfeits it.

> **Deliverables**
>
> `phase4/` (strip-mode attacker source, scripts for both runs) · logs/captures of Run A and Run B · report answers.

Phase 5 — Zone Walking [10 marks]

A signed zone must be able to *prove* that a name does not exist, and that proof is signed ahead of time — before anyone knows which name will be asked. A proof prepared in advance cannot be about the queried name itself, so it must reveal something about the names that *do* exist. Your task is to find out what it reveals and use it to reconstruct the zone's full contents, then close the leak.



5.1 Deploy

Reuse the Phase 3 signed, validating deployment. Populate `example.com` with at least five names of your choosing that are not advertised anywhere (e.g. internal service hostnames), and configure authenticated denial of existence so the walk below is possible. Exactly which signing option permits it is for you to discover.

5.2 Verify

Query a name you know does not exist and show how the zone answers for it.

5.3 Attack

- Starting from knowledge of `example.com` alone, recover the complete set of names using DNS queries only — no zone transfer (`AXFR`), no guessing from a wordlist. You should need no more than (number of names +2) queries.
- Then reconfigure denial of existence so the same procedure recovers *none* of the names, while a nonexistent name still gets an authenticated denial. Re-run the walk to show it now fails.

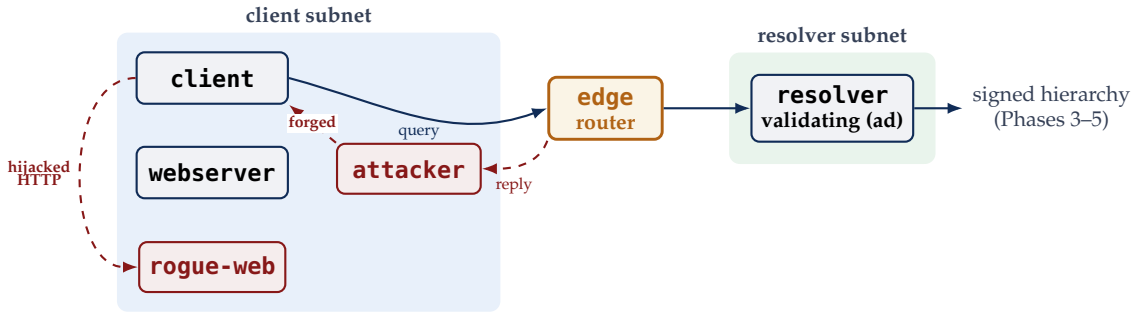
5.4 Report

- List the names you recovered and the exact steps and queries that produced them.
- Which configuration made the walk possible, and why does signed denial leak the contents when plain DNS does not?
- What does your fix cost the operator, and what can an enumerator still learn after it?

> **Deliverables**
>
> `phase5/` (the walkable zone and the fixed zone) · enumeration steps/commands and the recovered name list · evidence the walk fails after the fix · report answers.

Phase 6 — Last-Mile Response Forgery [15 marks]

The resolver validates, but the link from resolver to client carries no proof of it — the stub trusts whatever appears to come from its resolver. Move the adversary into the client subnet and forge the reply where DNSSEC's guarantee runs out.



6.1 Deploy

- Reuse the Phase 3 signed, validating deployment, upstream untouched. Relocate the `attacker` to `172.16.1.100` on the client subnet and keep `rogue-web` reachable.
- Divert the resolver → client responses through the attacker by **routing**: add a route on the `edge-router` that sends the client-bound replies via the `attacker`, which relays them on — no ARP spoofing, and the client's own configuration is untouched.

6.2 Verify

Relaying transparently, `client` still resolves `example.com` and fetches the real page.

6.3 Attack

Forge the resolver → client response for `example.com` to point at `rogue-web`, then fetch the site, against two clients: **Client A**, an ordinary stub (`curl` / the OS resolver); and **Client B**, a locally-validating client (e.g. `delv` or a validating stub) that holds the root key as its own trust anchor. The goal is to land Client A on the rogue page; record what Client B does and why.

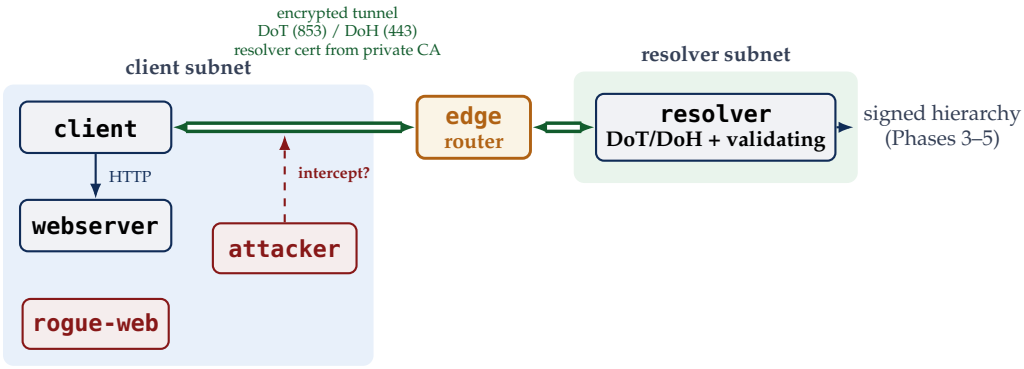
6.4 Report

- Report the outcome for each client, with logs and captures.
- DNSSEC validates upstream — why does it not stop this? Distinguish *where* validation happens from *where* trust is placed.
- What in a reply tells a stub its answer was validated (the `ad` flag), and why can an on-path attacker set it at will on the last mile? What does Client B do differently to catch the forgery?

> **Deliverables**
>
> `phase6/` (relocated attacker, downstream diversion scripts) · logs/captures for both clients · report answers.

Phase 7 — Encrypting the Last Mile (DoT / DoH) [15 marks]

Authenticate and encrypt the client–resolver link, then mount the last-mile attack again from three angles and account for each result.



7.1 Deploy

- Reuse the Phase 6 deployment. Stand up a private CA and install it in the client's trust store (no insecure bypass — `-k` or equivalent).
- Reach the resolver **by name, never by IP**. Give it the DNS name `resolver.cs6008.local`, add that on the client (e.g. the compose `extra_hosts`) pointing at the address where the client reaches the resolver, and issue the resolver's certificate with exactly that name as its `DNS` subject-alternative name (**no** IP SAN). The client then validates the certificate against the hostname it connected to.
- Enable DoT (TCP 853) and/or DoH (TCP 443) on the resolver and point the client at the encrypted transport by that name.

7.2 Verify

The client resolves over the encrypted transport with the certificate validating cleanly, upstream DNSSEC still holds, and a capture on `edge-router` shows no cleartext DNS for the lookup.

7.3 Attack

Mount three distinct attempts, one per aim: (i) alter the answer in transit, (ii) impersonate the resolver with a rogue certificate, and (iii) deny the encrypted transport — e.g. block TCP 853/443 — to force a cleartext fallback. Run each against a client that *refuses* cleartext and a client that *permits* it, and determine for each of the six (attempt × client) combinations whether it lands.

7.4 Report

- For each attempt and client, report whether it worked and explain why or why not.
- DoT or DoH — which did you deploy, and what are its drawbacks relative to the other?
- What does an encrypted, authenticated last mile still *not* protect?

> **Deliverables**
>
> `phase7/` (PKI scripts/certificates, DoT/DoH resolver config, both client transport configs) · logs/captures of encrypted resolution and of each attempt · report answers.